Chapter 3: Basing Software Development on Reusable Technology

3.0 Building on the Experience of Others

Why Build X When Y Exists?

Types of reuse:

- Expertise
- Standard designs and algorithms
- Libraries of classes or procedures
- Powerful commands built into languages and operating systems
- Frameworks
- Complete applications

3.1 Frameworks: Reusable Subsystems

A framework is reusable software that implements a generic solution to a generalized problem

Principle: Applications that do different (but related) things tend to have similar designs

3.1.0 Frameworks to promote reuse

A framework is inherently *incomplete*

- Certain classes or methods are used by the framework but are missing (slots)
- Some functionality is optional, requiring implementation by the dev
- Developers use services provided by the framework. **Services are called the API**

3.1.1 Object-oriented frameworks

In the object oriented paradigm, a framework is composed of a library of classes.

- The API is defined by the set of all public methods of these classes
- Some of the classes will normally be abstract and there are often many Interfaces

3.1.2 Frameworks and product lines

A product line (or product family) is a set of products built on a common base of technology

- The various products in the product line have different features to satisfy different markets
- The software common to all products is included in a framework
- Each product is produced by filling the available hooks and slots

3.1.3 Types of frameworks

- A *horizontal* framework provides general application facilities that a large number of applications can use
- A *vertical* framework (application framework) is more 'complete' but still needs some slots to be filled to adapt it to specific application needs

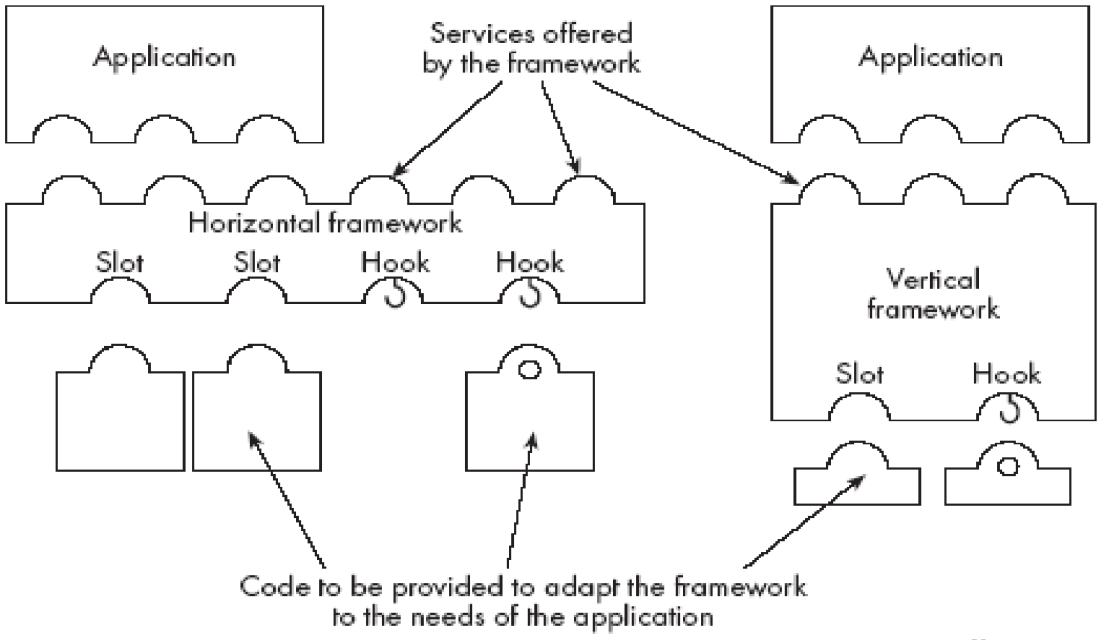


Figure 1: Horizontal vs Vertical Framework

3.2 The Client-Server Architecture

A *Distributed system* is a system in which:

- Computations are performed by separate programs
- processes normally run on separate machines
- machines and programs cooperate to perform the task of the system

Server:

- A program that provides a service for other programs that connect to it using a communication channel

Client:

- A program that accesses a server (or multiple) to obtain services
- Multiple clients may access a server simultaneously

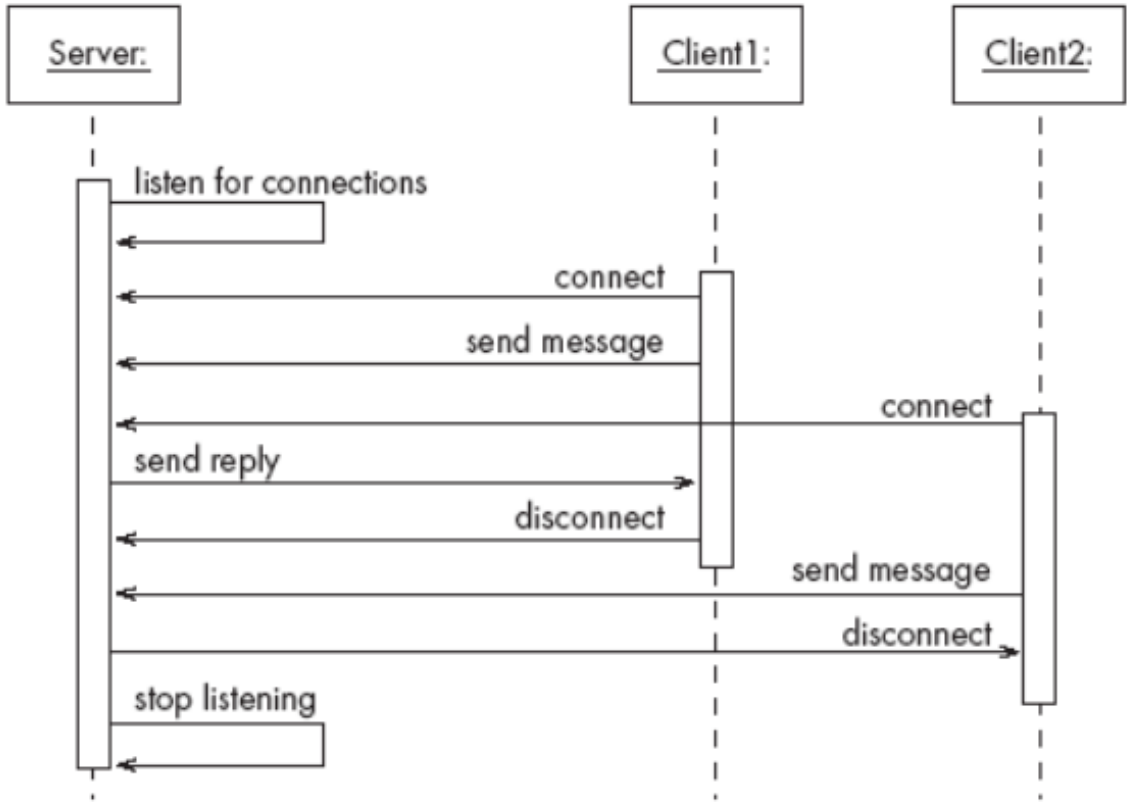


Figure 2: A server program communicating with two client programs

3.2.0 Activities of a server

1. Initializes itself
2. Starts listening for clients
3. Handles the following types of events originating from clients
   (a) accepts connections
   (b) responds to messages
   (c) handles client disconnections
4. May stop listening
5. Must cleanly terminate

3.2.1 Activities of a client

- Initialize the client and initiate a connection.
- Send messages and handle replies, server disconnection, and communication failures.
- Close resources and terminate cleanly.

3.2.2 Threads, work distribution, and protocols

- A server must accept connections while serving existing clients. OCSF separates the listening thread from client-connection threads.
- Thin clients delegate most processing to the server. Fat clients do more locally, potentially reducing server load.
- A protocol defines valid messages in each direction and rules for their exchange. Both sides must agree on content, order, and error handling.
- Design the application work, its distribution, messages, initialization, connection handling, and termination.
- Benefits include remote access, shared services, independent client/server development, and choices about central or distributed data.

3.3 Technology Needed for Client-Server Systems

- IP routes packets among hosts. TCP provides a reliable, ordered byte stream over a connection; it does not define application message boundaries.

- DNS resolves host names to addresses. localhost refers to the local machine, so a remote client needs the actual server address or host name.
- IPv4 addresses have 32 bits; IPv6 addresses have 128 bits.
- Clients need a server host and port. Port numbers range from 0 to 65535.
- System/well-known ports are 0-1023, registered/user ports are 1024-49151, and dynamic/private ports are 49152-65535. Actual ephemeral allocation depends on operating-system configuration.
- UDP sends datagrams without TCP's connection, reliable delivery, or ordering guarantees. It still has protocol overhead.

3.3.0 Connections and streams in Java

- java.net.ServerSocket listens for connections. new ServerSocket(port) binds a listening socket; accept() waits and returns a separate Socket for the connected client.
- new Socket(host, port) attempts a TCP connection. It does not continually ping until accepted; connection attempts can fail with an exception.
- getInputStream() receives bytes and getOutputStream() sends bytes. Stream classes belong to java.io.
- DataInputStream and DataOutputStream encode primitive values. Pair matching methods in the same order, such as writeUTF with readUTF.
- ObjectInputStream and ObjectOutputStream exchange serialized objects. Peers must agree on compatible message types.
- Close connections and streams when finished, and handle failures rather than assuming the peer remains connected.

3.3.1 Local example

- Client.java connects to localhost on port 8888, writes a UTF message, reads a UTF reply, and closes the socket.
- Server.java listens on port 8888, accepts one connection, reads a UTF message, writes a reply, and closes that connection.
- Serving multiple clients requires an accept loop and a strategy for handling each connection. Extending Thread alone does not start a background thread.

3.4 The Object Client-Server Framework (OCSF)

- OCSF supplies connection and communication behaviour through AbstractClient, AbstractServer, and ConnectionToClient.
- Extend intended abstract classes instead of modifying framework code.
- Services are methods application code calls. Slots are required methods subclasses implement. Hooks are optional callback methods subclasses may override.

3.5 The Client Side

- Subclass AbstractClient and implement handleMessageFromServer(Object msg).
- openConnection(), closeConnection(), and sendToServer(msg) control communication.
- isConnected(), getHost(), getPort(), and getInetAddress() expose state and configuration; setters configure host and port.
- connectionEstablished(), connectionClosed(), and connectionException(...) support lifecycle and error handling.
- Internally, the lecture's client has a Socket, object input/output streams, a receiving thread, and host/port information. Its run method receives messages and invokes callbacks.

3.6 The Server Side

- Subclass AbstractServer and implement handleMessageFromClient(Object msg, ConnectionToClient client).
- AbstractServer listens for connections; each ConnectionToClient represents one connected client.
- listen() starts listening. stopListening() stops accepting new connections while existing connections can remain active. close() also closes client connections.
- sendToAllClients(msg) broadcasts. ConnectionToClient.sendToClient(msg) sends to one client, and close() closes that connection.
- Accessors include isListening(), getClientConnections(), and getPort(). Configuration includes setPort() and setBacklog().
- Hooks include serverStarted, serverStopped, serverClosed, clientConnected, clientDisconnected, clientException, and listeningException.
- ConnectionToClient.setInfo and getInfo maintain application-specific information. The slides describe a HashMap, synchronized methods, a ThreadGroup, and periodic accept timeouts as

implementation details of their OCSF version.

3.7 SimpleChat

- EchoServer extends AbstractServer. Its message handler calls sendToAllClients, echoing incoming messages to connected clients.
- ChatClient extends AbstractClient. handleMessageFromClientUI forwards input using sendToServer; handleMessageFromServer calls the UI's display operation.
- ClientConsole implements ChatIF and provides console input/output. Using that interface lets a GUI replace the console without rewriting networking logic.
- Message flow: console input, ChatClient, server, connected ChatClients, then their displays.
- Separate UI responsibilities from communication to support replacement and reuse.

3.8 Risks of Reuse and Client-Server Development

- Poor reusable components create quality and support problems. Evaluate development practices, active support, and suitability before adopting them.
- Compatibility can break as components evolve. Prefer supported interfaces and plan for compatible versions.
- Reusable technology requires investment, testing, documentation, and support. Plan it as a product with users and requirements.
- Reduce not-invented-here resistance with quality, support, and responsiveness. Avoid divergent local modifications by providing general extension mechanisms.
- Network systems require attention to security and failures. Evolving clients and servers requires compatible protocols and migration planning.